DocFinder: Analysis and Design Document

Mobile Application Based on IEEE 1074 Software Development Standard

Quaid-i-Azam University, Islamabad

Department of Computer Science

Bachelor of Science in Computer Science

Prepared By: **Ahmad Masood**
Registration No: 04072213041

Prepared By: **M. Ahmad Hassan**
Registration No: 04072213018

Semester: 7th

November 4, 2025

Contents

1 Executive Summary / App Overview **2**

2 Software Development Selection Process (IEEE 1074) **2**

2.1 Project Justification 2

2.2 Alternatives Considered 2

3 Initiation **2**

3.1 Scope 2

3.2 Exclusions 2

4 Project Planning **2**

4.1 Roles 2

4.2 Schedule 3

5 Risk Management **3**

6 Pre-Development Processes **3**

6.1 Requirements Elicitation 3

6.2 Feasibility 3

6.3 Architecture 3

7 Requirements **3**

7.1 Functional Requirements 3

7.2 Non-Functional Requirements 4

8 Use Case: Find Nearest Open Doctor **4**

9 Design **4**

9.1 System Components 4

9.2 Data Model 5

1. Executive Summary / App Overview

DocFinder is a mobile application designed to assist users in locating nearby doctors, using GPS location and clinic working hours to show availability status (Open/Closed). It targets both rural and urban users needing fast, reliable access to medical assistance.

Platform: Android (Kotlin) **Primary Goal:** Quick, verified doctor lookup with real- time availability and distance display.

2. Software Development Selection Process (IEEE 1074)

2.1. Project Justification

There is a lack of reliable systems that can show doctor availability in rural areas. DocFinder solves this by using location and schedule logic.

2.2. Alternatives Considered

Web Application	Poor offline use, weak GPS support
Hybrid App	Performance limitations
Native Kotlin App (Chosen)	Best GPS + performance + offline capabilities

3. Initiation

3.1. Scope

Doctor profiles, GPS distance, working hours logic, filters, offline cache.

3.2. Exclusions

Online booking, payments, patient profiles.

4. Project Planning

4.1. Roles

Single-developer project performing PM, Analyst, Designer, Developer, Tester.

4.2. Schedule

Requirements	2 Weeks
Design	1 Week
Development	3 Weeks
Testing	2 Weeks
Deployment	1 Week

5. Risk Management

Unreliable GPS	Manual location option
Missing data	"Call for timings" fallback
API failure	Offline cache (Room DB)
Data privacy	Secure permissions + encryption

6. Pre-Development Processes

6.1. Requirements Elicitation

Interviews with patients, clinics. Identify required fields: name, timings, location, specialty, rating, fee.

6.2. Feasibility

Kotlin + Firestore + Google Maps + Room DB is feasible.

6.3. Architecture

Client-Server + Offline Cache

- Client: Kotlin app, UI, offline DB, GPS
- Server: Firebase (data + auth)
- Cache: Room DB
- Maps: Google Maps API

7. Requirements

7.1. Functional Requirements

1. Doctor profiles

2. Search: name, specialty, area
3. GPS distance calculation
4. Working hours + Open/Close logic
5. Filters (distance, rating, specialty, fee)
6. Manual location input
7. Offline caching

7.2. Non-Functional Requirements

- Performance: load < 3s online, < 8s offline
- Usability: simple clean UI
- Accessibility: readable fonts
- Security: encrypted API + secure permissions
- Reliability: 95% crash-free

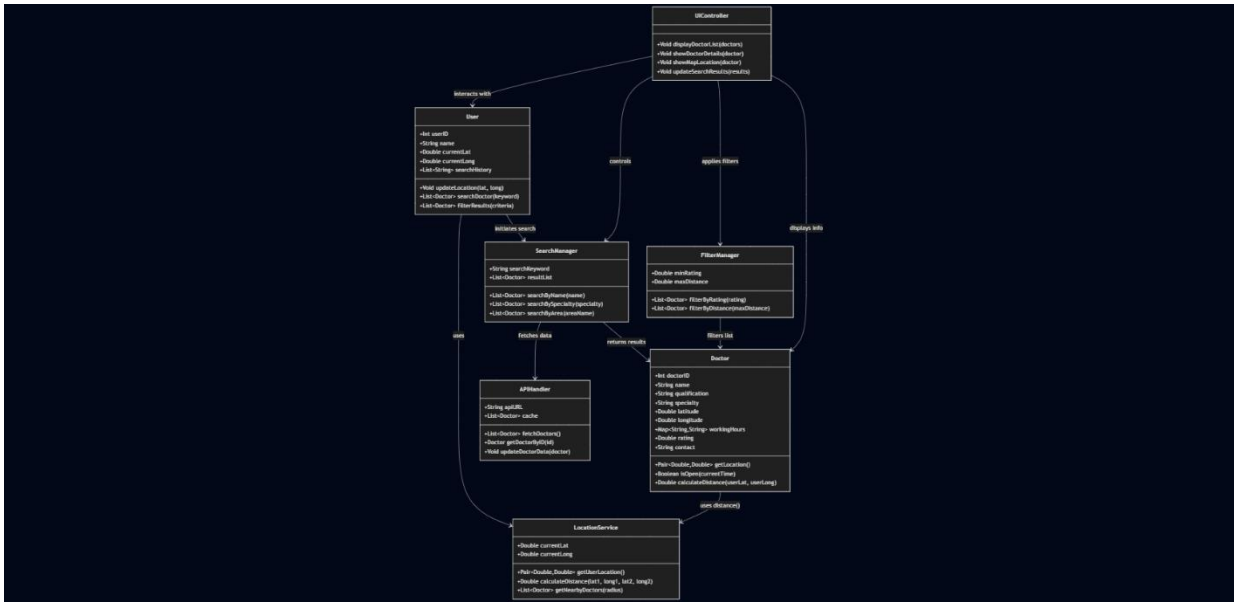
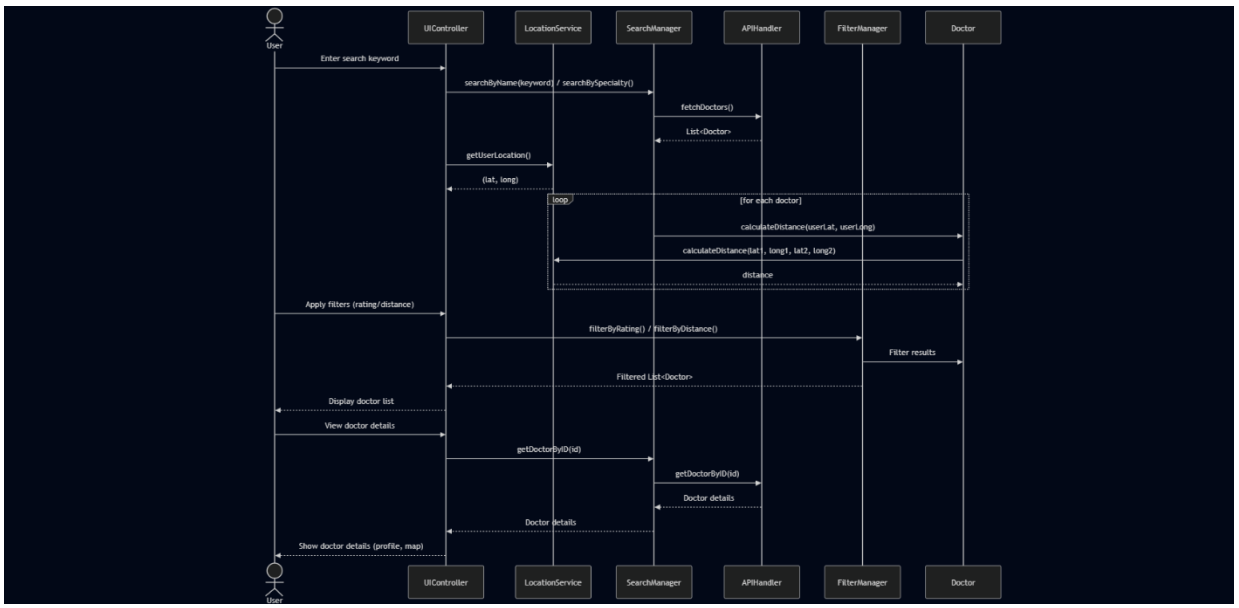
8. Use Case: Find Nearest Open Doctor

Flow: User opens app → app detects GPS / manual input → fetch doctors → sort by distance → display with Open/Closed status.

9. Design

9.1. System Components

UI (RecyclerView, Search bar, Filters) Business logic (Schedules, Geolocation) Data layer (Firestore + Room)

9.2. Data Model

DoctorID	Unique ID
Name	String
Qualification	String
Specialty	String
Clinic Name	String
Fee	Double
Rating	Double
Latitude, Longitude	Double
WorkingHours	JSON ({Mon: [09-13, 17-21]})
Phone	String